LedgerLens

Explainable financial entity resolution & payment intelligence

The idea in one sentence. LedgerLens helps a finance or data-operations analyst decide whether records from different systems refer to the same business, by combining transparent matching evidence with transaction signals and a human review step.

Executive summary

A business rarely appears in exactly the same way in every dataset. One system may store “ABC Technologies Pvt Ltd”, another “A.B.C. Technologies Private Limited”, and a payment file may contain only a shortened trading name. When these records are treated as unrelated, analysts lose the ability to build a reliable view of counterparties. When they are linked too aggressively, unrelated businesses can be combined and errors can spread into reporting or investigations.

LedgerLens addresses this identity-resolution problem with a deliberately cautious workflow. It standardizes records for comparison, generates candidate pairs, evaluates several pieces of evidence, and recommends whether a pair should be reviewed or investigated. It also surfaces simple payment-reference signals and shows supplied links in a relationship graph. It does not automatically merge entities or make fraud decisions.

1. The problem worth solving

The users

The initial users are finance operations analysts, data stewards and investigation teams who reconcile counterparties across customer, vendor, invoice and transaction datasets. Their task is often deceptively simple: determine whether two records represent the same real-world business. The work becomes difficult when identifiers are incomplete, names are abbreviated, legal suffixes vary, addresses are inconsistent or records arrive from different systems.

Why it matters

A false negative leaves one business fragmented across multiple identities; a false positive combines two different businesses. Both can distort downstream analysis. In a financial setting, the cost is not just a messy database: it can mean duplicated work, unreliable exposure summaries, missed relationships or an investigation based on a mistaken link. The right objective is therefore not “match as many records as possible”. It is “surface defensible candidate links and make uncertainty visible”.

Why this is an AI-era problem

More organizations are bringing together information from separate digital systems and document formats. Language models can help interpret unstructured descriptions, but financial identity decisions need stronger controls than a fluent answer. LedgerLens uses the idea of a hybrid system: probabilistic methods can help find candidates, while exact checks, transparent rules and human review govern consequential decisions.

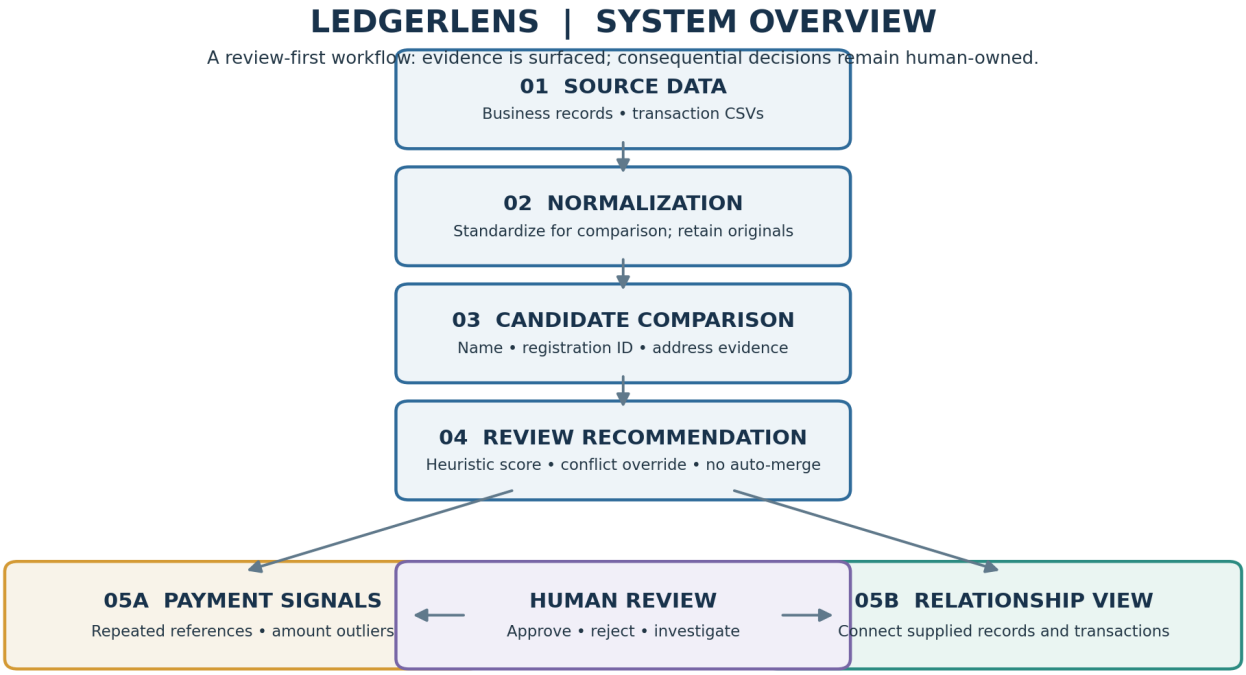
Initial scope

Included	Structured business records; business-name variants; registration identifiers; addresses; transaction references; review recommendations; relationship visualization.
Excluded	Real customer data; live bank or registry integrations; beneficial-ownership conclusions; sanctions screening; fraud determinations; automated payment actions.

DESIGN

2. How LedgerLens works

The workflow starts with structured records because they let us test the central identity-resolution problem without hiding errors inside document extraction. A later version could extract fields from invoices or remittance advice, but the extracted fields would still enter the same validation and review pipeline.



The six steps

01 Ingest	Load business records and transaction rows. Keep source IDs and original field values.
02 Normalize	Standardize case, punctuation and common legal suffixes for comparison; never overwrite the source value.
03 Compare	Generate candidate pairs and compare name similarity, registration identifiers and address similarity.
04 Explain	Return a score, field-level evidence and a conservative recommendation. Conflicting identifiers trigger investigation.
05 Connect	Flag repeated transaction references and simple amount outliers; show supplied record-to-transaction links.
06 Review	A human analyst approves, rejects or investigates. No pair is merged automatically.

Where AI belongs

A future document-ingestion module could use an LLM to extract candidate names, addresses, invoice references and payment explanations from messy text into a strict schema. The model should be required to abstain when a field is unclear and, where possible, return the source passage supporting each extracted value. It should not invent identifiers, infer legal identity from a name alone, calculate balances or approve a match. The current prototype does not yet implement this LLM extraction module; the working core is deterministic and testable.

Why not use an LLM for everything?

An LLM-only matcher would make it harder to reproduce decisions, reason about false matches and prove exactly which evidence drove a link. The better division of work is to use language understanding for messy inputs, conventional matching methods for candidate comparison, deterministic code for exact constraints and calculations, and a person for consequential approval.

3. Making a match explainable

The prototype compares three evidence groups. Name and address similarity are approximate; registration identifiers are compared exactly when both values are present. The score is a transparent ranking aid for review—not a probability that two records are the same legal entity.

Illustrative score	0.65 × name similarity + 0.25 × exact registration-ID match + 0.10 × address similarity

The current code uses illustrative score bands: 0.88 and above is labelled high similarity for review; 0.68–0.879 is a possible match for review; below 0.68 is not automatically linked. If both registration identifiers are present and differ, the system overrides the score and returns an identifier-conflict investigation recommendation. These weights and bands are design assumptions for a small demo. They must be calibrated on labelled data before any operational use.

Example: a pair of records has near-identical normalized names and the same registration identifier. The system can show that evidence and recommend high-priority review. If another pair has similar names but different non-empty registration identifiers, the conflict is surfaced instead of allowing the similarity score to dominate. The system does not silently merge either pair.

What an analyst should see

Evidence	Interpretation	Control
Name similarity	Useful for finding spelling and formatting variants; not unique identity proof.	Show original and normalized names.
Registration ID	Strong evidence when present and exact; a conflict matters.	Conflict sends pair to investigation.
Address similarity	Supporting context; addresses may be shared, old or incomplete.	Never use address alone to auto-link.
Payment reference	Repeated reference may indicate duplicate or repeated export.	Flag for review; do not call it fraud.

Keeping knowledge current and decisions traceable

The source records and matching rules—not the language model—are the system's reference point. A production version should preserve the original source record, source system, ingestion timestamp, effective date and rule version. Every analyst action should create an append-only review event containing the candidate pair, evidence shown, decision, reviewer and timestamp. If source data or rules change, affected links should be re-evaluated rather than silently carrying forward old conclusions.

Human review is a control, not a decoration

A reviewer should be able to accept a proposed link, reject it, or mark it unresolved. Conflicting identifiers, low-quality source data and ambiguous cases should be routed to investigation. Any future use of approved links in downstream reports should preserve the fact that the link was human-approved and record which evidence was available at the time.

4. What has been built, what remains to prove

The accompanying ZIP contains a runnable Streamlit application and Python modules. The included synthetic dataset has 10 business records and 8 transaction rows. The dashboard shows pairwise comparisons and their evidence, payment-reference and amount signals, and a graph connecting supplied business records to transactions.

Prototype components

Component	Implementation	Current boundary
Entity comparison	Python normalization and string similarity; exact ID check.	Small all-pairs demo; not production-scale.
Payment signals	Repeated transaction-reference flag and median-absolute-deviation outlier flag.	Signals only; not fraud detection.
Relationship graph	NetworkX graph of supplied record-to-transaction links.	Does not infer ownership or control.
User interface	Streamlit tables, pair inspection and graph view.	Local demo; no production access control.

Test evidence

12 automated unit tests passed in the included local test run. The tests check suffix normalization, matching records with the same registration ID, conflicting identifiers, the no-auto-merge safeguard, empty-name handling, duplicate-reference flags, graph nodes and edges, and score bounds. The exact test command and output are included in **docs/test_results.txt**.

This is evidence that selected code paths behave as expected on the supplied synthetic examples. It is not a benchmark of real-world matching quality. The dataset is small and hand-constructed; it cannot establish precision, recall, false-match rate, business savings or fraud-detection performance.

How I would evaluate the next version

Measure	What it tells us	How to test it
Precision of accepted links	How many proposed links are correct.	Use independently labelled entity pairs; report false positives.
Recall	How many true same-entity pairs are found.	Measure missed links against labelled ground truth.
Review rate	How much work remains for analysts.	Track auto-abstentions and human review volume.
Calibration	Whether confidence values correspond to observed correctness.	Calibrate only after collecting representative labels.

Risks, mitigations and weakest point

Risk	Mitigation / remaining work
False links	No auto-merge; expose evidence; use identifier conflict override; evaluate false positives on labelled data.
Missing or outdated data	Retain provenance and effective dates; show freshness; reprocess affected records when sources change.
LLM extraction errors (future)	Schema validation, source-span evidence, abstention, field-level review and no invented identifiers.
Privacy and liability	Synthetic-only demo; production requires approved data handling, access controls, retention policy and accountable human sign-off.

The weakest point today is the evidence base: the score is uncalibrated, the address comparison is simplistic and the synthetic sample is too small to represent real operational data. Before investing further, I would speak with finance operations practitioners, build a representative labelled dataset, agree the cost of false positives versus false negatives, validate privacy and data-access constraints, and test whether the proposed workflow fits the team's actual review process.

APPENDIX

What I added beyond AI

A transparent account of AI assistance, corrections and the engineering decisions behind the prototype.

How AI was used

I used an LLM as a working assistant during problem exploration and solution design: to compare possible finance problems, challenge the proposed architecture, help organize the written explanation, and generate an initial implementation and test ideas. I treated its suggestions as drafts rather than authoritative answers. I then kept the prototype deliberately small so its matching rules and limitations could be inspected and tested.

Representative prompts from the work

- "Compare finance operations problems where entity resolution could reduce manual work. Explain the users, data involved, failure costs and what a small prototype could demonstrate."
- "Design a hybrid entity-resolution workflow. Separate approximate similarity from exact identifier checks and human approval. Identify where a false positive could cause downstream harm."
- "Review the prototype's matching logic and synthetic test cases. Find edge cases, explain what the tests do not prove, and suggest a defensible evaluation plan."

Suggestions I did not accept at face value

- A high name-similarity score is not proof of identity. I kept the score as a review aid and added an explicit investigation path for conflicting non-empty registration identifiers.
- An LLM should not make the final entity-linking decision. The prototype does not automatically merge records; a reviewer remains responsible for consequential decisions.
- A duplicate transaction reference or unusual amount is not proof of fraud. The dashboard describes these as review signals only.
- Passing unit tests does not establish real-world accuracy. The report separates code-level tests from the future need for a labelled evaluation dataset.

Engineering decisions to be able to defend

- Preserve raw source values and source IDs while creating normalized comparison fields, so every proposed link can be traced back to its input.
- Keep approximate matching, exact identifier checks, recommendations and human decisions as separate layers.
- Use a simple Python implementation first. Defer multi-agent orchestration and production graph infrastructure until the problem and scale justify them.
- Treat uncertainty as a valid outcome: investigate or abstain rather than forcing a match.

One design principle I can explain clearly

Entity resolution is not just a text-similarity problem. Similar names can belong to different legal entities, and the same entity can appear under different names. A similarity score helps prioritize candidates, but it does not prove identity. Exact identifiers provide stronger evidence when reliable; conflicting identifiers should stop an automatic link; and a person should approve decisions that could affect downstream financial analysis. This is why LedgerLens separates candidate discovery from identity decisions.

Submission integrity note

The current prototype is a deterministic structured-data demonstration. LLM-based document extraction, production integrations, calibrated matching probabilities and real-world performance evaluation are not implemented. The included test results are from the synthetic prototype only. Before submission, I should be ready to explain the code, reproduce the tests and accurately describe which parts were AI-assisted.